

Memory Architecture — Part 2 · Story Edition

Scaling & sustaining your agent's memory · 5 videos · ≤3 min each

Style note — each script follows **one person, Ali, a statistician**, across his real weekly workflow, and goes *deep on a single example* rather than listing several. Every script still **leads with the answer** (Minto's Pyramid Principle), frames the problem as **Situation → Complication → Question → Answer** (SCQA), keeps its teaching points **MECE** (no overlap, no gaps), uses **one metaphor per concept**, and ends with one small file to create. The continuous Ali story carries across all five videos. Text below is the word-for-word voiceover.

SLO coverage map — which video covers which SLO

Video	Name	SLO covered
Video 1	The Skill File — Your Agent's Standard Operating Procedure	Create a well-structured skill file
Video 2	Skills vs. Agents — Knowing What to Build	Difference between a skill file and agents
Video 3	Weekly Maintenance — Protecting Your Memory's ROI	Perform weekly maintenance
Video 4	Fresh Sessions — Resetting Without Losing Ground	Start fresh sessions strategically
Video 5	Lazy Loading — Scaling Memory Without Drowning In It	Implement lazy loading for large memory

The Skill File — Your Agent's Standard Operating Procedure

SLO: Create a well-structured skill file

~2:30

Teaches: a skill file is an SOP for one task; the four parts (name · when to use · steps · one example); the one-skill-one-job rule.

PYRAMID LEAD (answer first)

The fastest way to make your agent — that is, your project — more reliable is to stop re-explaining tasks and start writing skill files.

The fastest way to make your agent — that is, your project — more reliable is to stop re-explaining tasks... and write one skill file. Lead with that — everything else follows.

Meet Ali. Ali is a statistician at a research lab. Every Monday, a new dataset lands on his desk, and his job is to turn it into one clean summary report — the same shape, every week.

So every Monday, Ali opens his agent and explains the task. "Drop the empty rows. Check the totals. Flag anything that looks off. Then write it up in our standard format." The results are good... but he types that same paragraph again, and again, week after week.

One Monday he stops, and asks a sharper question. Why am I teaching this task from scratch every single time? That question is the whole lesson.

The fix is a skill file. Think of it as a standard operating procedure — an SOP — for one task. A short, structured document that teaches your agent to do that one task the same way, every time. Write it once. Reuse it forever.

Here's what Ali actually wrote — and it's only four parts.

One: a **name**. In plain words — "Clean and summarize a weekly dataset." If he can't name it simply, the task isn't clear yet.

Two: **when to use it**. "Use this whenever a new raw dataset arrives." Now the agent reaches for it at the right moment, without Ali asking.

Three: the **steps**. Short, and in order. Drop the empty rows. Verify the column totals. Flag the outliers. Write the summary in the house format. Four lines — not an essay.

Four: **one worked example**. Ali pastes in last week's finished report. Now "good" is visible — the agent can see the target, instead of guessing at it.

Name. When. Steps. Example. Nothing more.

Now, the trap Ali nearly fell into. He was tempted to pour everything into one giant file — cleaning, charting, emailing the team, all at once. Don't. The rule is **one skill, one job**. A bundled file turns messy... and a messy file gets ignored. When Ali needed charts, he simply wrote a second skill.

And his very first draft? Honestly — rough. And that's exactly right. An SOP earns its quality through use; Ali sharpened it the next Monday, and the Monday after that. A working draft beats a perfect file you never wrote.

So here's your move. Pick the one task you repeat most — the paragraph you keep retyping. Then ask: if I had to write its SOP — its name, when to use it, the steps, and one example — what would it say? Write that down. That... is your first skill file.

 **YOUR FILE:** One skill file for a task you repeat — with all four parts (name · when to use · steps · one worked example). Keep it to a single task.

Skills vs. Agents — Knowing What to Build

SLO: Difference between a skill file and agents

~2:25

Teaches: a skill = an instruction; an agent = the operator that uses many skills and judges; when to build each; skills compound the agent's value.

PYRAMID LEAD (answer first)

A skill is an instruction; an agent is the operator. Confuse the two and you'll over-build.

A skill is an instruction; an agent is the operator. Confuse the two... and you'll over-build everything. So let's be precise, because the distinction changes what you build.

Back to Ali. He now has a shelf of skills — clean a dataset, build a chart, draft a summary. But he hits a new confusion: what exactly is the difference between a skill and the agent itself?

Picture Ali's own desk. The skill is the printed procedure clipped above it — fixed steps for one job. Ali, the statistician, is the operator — he runs many procedures, watches for the odd result, and decides what to do when a number looks wrong. The procedure can't act. Ali acts, using it.

So a skill is know-how. An agent is the doer that puts know-how to work.

When should Ali build each? Two clean cases, no overlap. If he wants one task done the same way every time — that's a skill. If he needs something to take initiative across many steps, choosing and combining skills as it goes — that's an agent.

And here's the relationship that ties it together: you equip an agent with skills. A strong agent is simply a capable operator with a shelf of sharp skills. This is where the return compounds — improve one skill, and every task that uses it gets better, with no change to the agent itself.

So Ali builds a content agent for his weekly reports. He hands it three skills — clean the data, build the chart, draft the summary. The agent decides which to use for each new dataset, runs them in order, and hands him a finished pack. He wrote the skills once; the agent reuses them every week.

If the line still feels blurry, that's normal — it sharpens the moment you build one of each.

So take three tasks you do often, and sort them. Does this just need to run the same way every time? That's a skill. Or does it need to take charge across many steps? That's an agent. That one sort tells you exactly what to build next.

 **YOUR FILE:** List 3 tasks you do often. Mark each **skill** (same way every time) or **agent** (takes charge across many steps), with one line on why.

Weekly Maintenance — Protecting Your Memory's ROI

SLO: Perform weekly maintenance

~2:15

Teaches: memory decays without upkeep; a 15-minute weekly routine of 4 steps — prune journal · trim handbook · close tasks · archive stale files.

PYRAMID LEAD (answer first)

An unmaintained memory system decays — a 15-minute weekly review protects everything you built.

Let's start with the conclusion: an unmaintained memory system quietly decays... and a fifteen-minute weekly review protects everything you've built. That's the whole video; the rest is why and how.

Ali has built a lot now — a handbook, a journal, a shelf of skills, a task list. But systems drift. Over a few weeks, his journal fills with notes that are no longer true. A handbook rule expires. Finished tasks sit open on the list.

Left alone, his agent starts making decisions from outdated information — and Ali won't notice until a report comes out wrong.

Think of it like the instruments in Ali's lab. A calibrated instrument left unchecked drifts — slowly, invisibly — until one day the readings are simply wrong. Nobody schedules that failure; they just skip the calibration. Memory drifts the same way.

So Ali runs four steps, once a week. One: skim the journal — keep what's still true, let the rest go. Two: open the handbook — trim a rule that no longer fits, add one he keeps needing. Three: work the task list — close what's done, note what's next. Four: archive or delete the files he's finished with. That's the whole routine — fifteen minutes.

For Ali, it's every Friday. He opens three things — his `CLAUDE.md`, his `memory.md`, and his task tracker. He reads each quickly, tidies as he goes, closes anything finished. Come Monday, his agent reads only current information — not last month's leftovers.

An honest word: Ali will miss a week. That's fine — maintenance isn't a perfect streak, it's a habit you return to. Even one pass after two weeks away beats no habit at all.

So choose your slot now — Friday afternoon, say. Set a fifteen-minute timer, and ask the only question that matters: if I read my own files right now... is everything still true? Or am I about to carry clutter into next week?

 **YOUR FILE:** A `maintenance.md` checklist (the four steps), a chosen weekly day, and one full pass done this week — noting what you cleaned up.

Fresh Sessions — Resetting Without Losing Ground

SLO: Start fresh sessions strategically

~2:25

Teaches: a full context degrades performance; a reset restores it without losing memory (memory lives in files); 3 triggers; save-to-file first.

PYRAMID LEAD (answer first)

When an agent slows or drifts, the fix usually isn't a better prompt — it's a fresh session.

When your agent slows down, drifts, or starts repeating itself, the instinct is to write a better prompt. Usually the real fix is simpler: start a fresh session. Lead with that.

One afternoon, Ali's agent starts dragging. It repeats a question he already answered. Its replies turn vague. His first instinct is to craft a cleverer prompt — but the real problem is somewhere else.

Here's the mechanism. Your agent works on a desk — its short-term attention, the space it's actively holding right now. A long session piles that desk high with old questions, earlier answers, and a couple of wrong turns. The fuller the desk, the slower and foggier the thinking. A fresh session clears the desk — and, crucially, Ali loses nothing, because the real memory lives in his files, not the chat.

Think of Ali's whiteboard. Six hours into a hard analysis, it's a tangle of crossed-out numbers and dead ends. Wiping it clean isn't losing the work — the conclusions are already saved in his notebook. The clean board is what lets him think clearly again.

So when does Ali reset? Three triggers, cleanly separated. One: he's switching to a different task — finish, then start clean. Two: the conversation has started going in circles and isn't improving. Three: the answers are getting sloppy — the tell-tale sign the desk is overloaded.

One discipline before he resets: save what matters to a file first. Reached a decision? Write it to the decision log. Learned something? Add it to the journal. Then the new session opens with the good material already on the shelf.

So Ali spends an hour getting a tricky analysis exactly right. He saves the final notes to `memory.md`, records the key choice in `decision.md` — then starts fresh to write the report. The new session is fast and sharp, and nothing valuable was lost.

It can feel wasteful to walk away from a conversation you put real effort into. But you're not discarding the work — only the clutter. The work is safe in your files. So next time things feel slow or stuck, ask one question: have I saved what matters? If yes — reset, and feel the difference.

 **YOUR TASK:** Next time you switch tasks, save what matters to a file, then start a fresh session. Note in `memory.md` when the reset helped.

Lazy Loading — Scaling Memory Without Drowning In It

SLO: Implement lazy loading for large memory

~2:30

Teaches: loading all memory hurts speed & cost; keep a small always-loaded index and load detail on demand; the 3 steps (split by topic · one-line index · load on demand).

PYRAMID LEAD (answer first)

As memory grows, loading everything every time hurts speed and cost — load an index always, details on demand.

Start with the answer: as your agent's memory grows, loading all of it every time makes the agent slower and more expensive — so load a small index always, and the details only when you need them. That's lazy loading.

Ali's system is working — so well that his memory keeps growing. The situation is a good one: his agent keeps learning, so the notes keep piling up. The complication is the catch — attention is limited, and the desk only holds so much.

Pile a hundred pages of notes onto that desk, and the important things get buried — the agent slows, and every run costs more. More memory is not more useful. The right memory, at the right moment, is.

Picture Ali's filing room. He doesn't carry every folder to his desk each morning — he keeps one index card listing where each folder lives, and pulls a folder only when a question needs it. That's what keeps him fast.

For Ali's agent, it's three steps. One: split a big memory file into smaller ones by topic — decisions in one, lessons in another. Two: write a one-line index — a short file naming what each file is for. Three: keep only that index loaded by default; the agent opens a detailed file just when the task calls for it.

So Ali keeps a short `memory-index.md`. It reads: "decisions — why we chose things; lessons — what went wrong before; data-notes — where the numbers live." Ask about a past choice, and the agent reads the index, opens only `decision.md`, and leaves the rest on the shelf. Faster, sharper, cheaper.

If splitting files and writing an index sounds like a project, don't make it one — start with a single index... and one file split. You'll feel the difference immediately, and grow it from there.

So look at your largest memory file and ask: could this become a short index — plus a few smaller files I open only when I need them? That... is the whole move.

 **YOUR FILES:** A `memory-index.md` (one line per file, saying what it's for), and one oversized file split into smaller topic files the agent opens on demand.